

Experiment #02

Linux Shell commands for file operations and their rights

Objective

The main objectives of this lab are to understand:

1. How to view or edit a text file in Linux.
2. How we can concatenate two or more files into a single file,
3. How to sort contents of file
4. How to display specific lines of a file
5. How to assign different permissions to the file

using commands in Ubuntu operating system.

Software Tools

- Ubuntu 16.04
- GCC Compiler

Theory

Viewing and editing a Text File

- Using touch to **create** a blank text **file**: \$ touch NewFile.txt.
- Using cat to **create** a new **file**: \$ cat NewFile.txt. ... (write anything in your file CNTR+D to save and CTRL+Z to exit
- Simply using > to **create** a blank text **file**: \$ > NewFile.txt.

\$ cat [option] [filename]

i. \$ cat filename

Purpose: It will display the contents of the file *filename*.

ii. \$ cat > file1

Purpose: “>” is called output redirection operator. It permits user to write in *file1*. If *file1* already exists then it over writes the contents of the *file1*.

E.g. write any word/phrase after giving command **\$ cat > file1** like “Success is not a destination.”

Press **[Ctrl+d]**

Note: The above command creates the file called *file1* if *file1* does not exist and allow user to write text there. Press Ctrl+d (To save and exit).

iii. \$cat >> file1

Purpose: “>>” is use to append data in a file. It does not overwrite text of *file1* rather appends text at the end of *file1*.

E.g. write any word/phrase after using command **\$ cat >> file1** like “It’s a progressive journey. ”

Press **[Ctrl+d]**

Note: The above command appends data in *file1*. Here you can find that data is appended in file1 or not by using command **\$ cat filename**, which will display all the content of *file1*.

\$ cat file1

iv. **\$cat file1 file2 >file3**

The above command is used to write contents of the file1 and file 2 into file3

v. **\$ head [option] Filename**

Purpose: “head” displays the top part of a file. By default, it shows the first 10 lines.

head -n allows you to change the number of lines to be shown.

- **\$ head File1:** It displays first 10 lines of *File1*.
- **\$ head -n50 file.txt:** Displays the first 50 lines of the *file.txt*
- **\$head -50 file.txt:** Produces the same result as above command and Displays the first 50 lines of the *file.txt*

vi. **\$ tail [option] Filename**

Purpose: “tails” displays the bottom portion of a file. By default, it shows the last 10 lines.

Same as head command.

tail -n allows you to change the number of lines to be shown.

- **\$ tail File1:** It displays last 10 lines of *File1*.
- **\$ tail -n15 file.txt:** Displays the last 15 lines of the *file.txt*

Redirection of Input and Output:

Mostly all command gives output on screen or take input from keyboard, but in Linux (and in other OSs also) it's possible to send output to file or to read input from file.

Linux redirection feature is used to detach the default files from stdin, stdout, and stderr and attach other files with them.

There are three main redirection symbols >, >>, <.

- **Input Redirection:**

command < input-file **OR** command 0< input-file

Purpose: Detach keyboard from stdin and attach 'input-file' to it, i.e., 'command' reads input from 'input-file' and not keyboard. . E.g. To take input for cat command **\$ cat <myfiles**

- **Output Redirection:**

command> output-file **OR** command 1> output-file

Purpose: Detach the display screen from stdout and attach 'output-file' to it, i.e., 'command' sends output to 'output-file' and not the display screen. E.g. **\$ ls** command gives output to screen; to send output to file of **ls** command.

Give command : **\$ ls> filename**

It means put output of ls command to filename. Now if *filename* file exist in your current directory it will be overwritten without any type of warning.

>> Redirector Symbol (Append)

Syntax:

command >> filename

Purpose: To output Linux-commands result (output of command or shell script) to END of file. Note that if file exist, it will be opened and new information/data will be written to END of file, without losing previous information/data, and if file is not exist, then new file is created.

E.g. To send output of date command to already exist file give command
\$ date >>myfiles

- **Error Redirection:**

command 2> error-file

Purpose: Detach the display screen from stderr and attach 'error-file' to it, i.e., error messages are sent to 'error-file' and not the display screen

Example of Input and Output Redirectors

You can also use above redirectors simultaneously as follows

Description "cat" is short for concatenate. This command is used to view and concatenate files.

Examples cat /etc/passwd

This command displays the "/etc/passwd" file on your screen.

cat /etc/profile

Department of Computer Engineering
HITEC University Taxila

This command displays the "/etc/profile" file on your screen. Notice that some of the contents of this file may scroll off of your screen.

cat file1 file2 file3 > file4

This command combines the contents of the first three files into the fourth file.

Create text file *sname* as follows:

```
$ cat>sname
```

```
virk
```

```
ash
```

```
zebra
```

```
babu
```

Press CTRL + D to save.

Sort the file

Now issue following command.

```
$ sort <sname>sorted_names
```

```
$ cat sorted_names
```

```
ash
```

```
babu
```

```
virk
```

```
zebra
```

Note: In above example sort (**\$ sort <sname>sorted_names**) command takes input from *sname* file and output of sort command (i.e. sorted names) is redirected to *sorted_names* file.

File Permissions:

Linux is a clone of UNIX, the **multi-user operating system** which can be accessed by many users simultaneously. Linux can also be used in mainframes and servers without any modifications. But this raises security concerns as an unsolicited or **malign user** can **corrupt, change or remove crucial data**. For effective security, Linux divides authorization into 2 levels.

1. Ownership
2. Permission

Ownership of Linux files

Every file and directory on your Unix/Linux system is assigned 3 types of owner, given below.

User

A user is the owner of the file. By default, the person who created a file becomes its owner.

Hence, a user is also sometimes called an owner.

Group

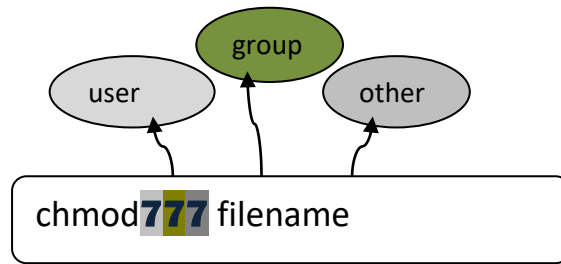
A user- group can contain multiple users. All users belonging to a group will have the same access permissions to the file. Suppose you have a project where a number of people require access to a file. Instead of manually assigning permissions to each user, you could add all users to a group, and assign group permission to file such that only this group members and no one else can read or modify the files.

Other

Any other user who has access to a file. This person has neither created the file, nor he belongs to a user group who could own the file. Practically, it means everybody else. Hence, when you set the permission for others, it is also referred as set permissions for the world.

Now, the big question arises how does **Linux distinguish** between these three user types so that a user 'A' cannot affect a file which contains some other user 'B's' vital information/data. It is like you do not want your colleague, who works on your Linux computer, to view your images. This is where **Permissions** set in, and they define **user behavior**.

Let us understand the **Permission system** on Linux.



Numerical permissions

#	Permission	rwX
7	full	111
6	read and write	110
5	read and execute	101
4	read only	100
3	write and execute	011
2	write only	010
1	execute only	001
0	none	000

Permissions

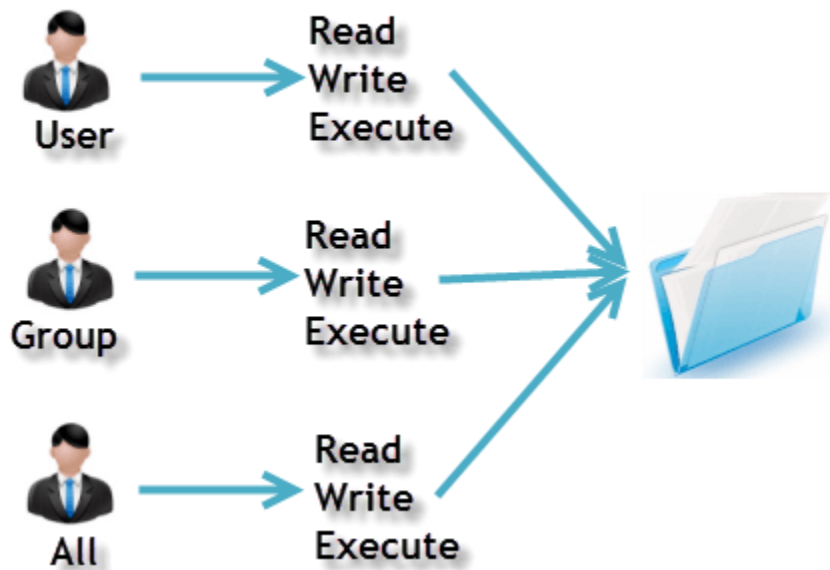
Every file and directory in your UNIX/Linux system has following 3 permissions defined for all the 3 owners discussed above.

- **Read:** This permission give you the authority to open and read a file. Read permission on a directory gives you the ability to lists its content.
- **Write:** The write permission gives you the authority to modify the contents of a file. The write permission on a directory gives you the authority to add, remove and rename files stored in the directory. Consider a scenario where you have to write permission on file but do not have write permission on the directory where the file is stored. You will be

able to modify the file contents. But you will not be able to rename, move or remove the file from the directory.

- **Execute:** In Windows, an executable program usually has an extension ".exe" and which you can easily run. In Unix/Linux, you cannot run a program unless the execute permission is set. If the execute permission is not set, you might still be able to see/modify the program code(provided read & write permissions are set), but not run it.

Owners assigned Permission On Every File and Directory



ls -l on terminal gives

ls -l

File type and Access Permissions.

```

home@VirtualBox: ~
home@VirtualBox:~$ ls -l
-rw-rw-r-- 1 home home 0 2012-08-30 19:06 My File
  
```

Here, we have highlighted '-rw-rw-r--' and this weird looking code is the one that tells us about the permissions given to the owner, user group and the world.

Here, the first '-' implies that we have selected a file.p>

`-rw-rw-r--`



indicates
file

Else, if it were a directory, **d** would have been shown.

`drwxr-xr-x 2 ubuntu ubuntu 80 Sep 6 07:27 Desktop`

The characters are pretty easy to remember.

r = read permission

w = write permission

x = execute permission

- = no permission

Let us look at it this way.

The first part of the code is '**rw-**'. This suggests that the owner 'Home' can:

`-rw-rw-r--`



no execute
permission

- Read the file
- Write or edit the file
- He cannot execute the file since the execute bit is set to '-'.

By design, many Linux distributions like Fedora, CentOS, Ubuntu, etc. will add users to a group of the same group name as the user name. Thus, a user 'tom' is added to a group named 'tom'.

The second part is '**rw-**'. It for the user group 'Home' and group-members can:

- Read the file
- Write or edit the file

The third part is for the world which means any user. It says '**r--**'. This means the user can only:

- Read the file



Changing file/directory permissions with 'chmod' command

Say you do not want your colleague to see your personal images. This can be achieved by changing file permissions.

We can use the '**chmod**' command which stands for 'change mode'. Using the command, we can set permissions (read, write, execute) on a file/directory for the owner, group and the world. **Syntax:**

`chmod permissions filename`

There are 2 ways to use the command -

1. **Absolute mode**
2. **Symbolic mode**

Absolute(Numeric) Mode

In this mode, file **permissions are not represented as characters but a three-digit octal number**.

The table below gives numbers for all for permissions types.

Number	Permission Type	Symbol
0	No Permission	---
1	Execute	--X
2	Write	-W-
3	Execute + Write	-WX
4	Read	r--
5	Read + Execute	r-X
6	Read +Write	rw-
7	Read + Write +Execute	Rwx

Let's see the chmod command in action.

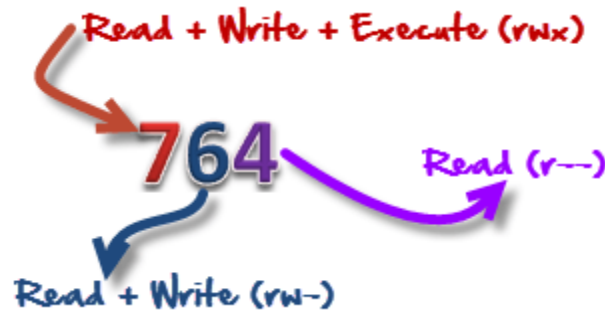
Checking Current File Permissions

```
ubuntu@ubuntu:~$ ls -l sample
-rw-rw-r-- 1 ubuntu ubuntu 15 Sep  6 08:00 sample
```

chmod 764 and checking permissions again

```
ubuntu@ubuntu:~$ chmod 764 sample
ubuntu@ubuntu:~$ ls -l sample
-rwxrw-r-- 1 ubuntu ubuntu 15 Sep  6 08:00 sample
```

In the above-given terminal window, we have changed the permissions of the file 'sample' to '764'.



'764' absolute code says the following:

- Owner can read, write and execute
- User group can read and write
- World can only read

This is shown as '-rwxrw-r-'

This is how you can change the permissions on file by assigning an absolute number.

Symbolic Mode

In the Absolute mode, you change permissions for all 3 owners. In the symbolic mode, you can modify permissions of a specific owner. It makes use of mathematical symbols to modify the file permissions.

Operator	Description
+	Adds a permission to a file or directory
-	Removes the permission
=	Sets the permission and overrides the permissions set earlier.

The various owners are represented as -

User Denotations	
U	user/owner
G	Group
O	Other
A	All

We will not be using permissions in numbers like 755 but characters like rwx. Let's look into an example

```

Current File Permissions
home@VirtualBox:~$ ls -l sample
-rw-rw-r-- 1 home home 55 2012-09-10 10:59 sample

Setting permissions to the 'other' users
home@VirtualBox:~$ chmod o=rwx sample
home@VirtualBox:~$ ls -l sample
-rw-rw-rwx 1 home home 55 2012-09-10 10:59 sample

Adding 'execute' permission to the usergroup
home@VirtualBox:~$ chmod g+x sample
home@VirtualBox:~$ ls -l sample
-rw-rwxrwx 1 home home 55 2012-09-10 10:59 sample

Removing 'read' permission for 'user'
home@VirtualBox:~$ chmod u-r sample
home@VirtualBox:~$ ls -l sample
--w-rwxrwx 1 home home 55 2012-09-10 10:59 sample

```

Example

\$chmod 750 filename

- ✓ Gives the user read, write and execute (full permission).
- ✓ Gives group members read and execute.
- ✓ Gives others no permissions.

Using numeric representations for permissions:

r = 4; w = 2; x = 1; total = 7

Lab Tasks

1. Show **Simulation** results after performing the following tasks by using suitable commands in Linux. Also show command/commands used for specific task
 - a. Create a blank text file on your home directory with your name.
 - b. Make a directory with name **student**.
 - c. Change the directory to **student** and copy your text file in this directory.
 - d. Assign no permission to anyone.
 - e. Verify that above task has done successfully.
 - f. Now try to write in this file. If error occurs, state why?
 - g. Assign permission of write only to user
 - h. Verify that above task has done successfully.
 - i. Write your introduction in 5 to 7 lines in .txt file.
 - j. Display contents of the file. If error occurs, state why?
 - k. Assign permission of read only to user
 - l. Verify that above task has done successfully.
 - m. Display contents of the file.
 - n. Assign permissions of read and execute only to user
 - o. Verify that above task has done successfully.
 - p. Assign permissions of read and write only to user
 - q. Verify that above task has done successfully.

- r. Assign all permissions to user
 - s. Verify that above task has done successfully.
 - t. Assign permission of read and execute only to group
 - u. Verify that above task has done successfully.
 - v. Assign all permissions to all users
 - w. Verify that above task has done successfully.
2. Show **Simulation** results after performing the following tasks by using suitable commands in Linux. Also show command/commands used for specific task
- a. Create 3 empty text files and name them as **students.txt**, **pstudent.txt** and **fstudents.txt**.
 - b. Assign permissions to read only permission to the user.
 - c. Enter at least 10 10 different students' names in **pstudent.txt** and **fstudent.txt**. If any error appears, debug that error for user only.
 - d. Now create directory named **HITEC** and assign permissions of read only to user and group.
 - e. Copy all files to directory **HITEC**. If any error appears, debug that error for user and group only.
 - f. After that append the file **students.txt** with first five sorted names from **pstudent.txt** and last five sorted names from **fstudent.txt**.
 - g. Show the contents of sorted names from file **students.txt**.
 - h. Change the permissions of file **students.txt** read only and both other files read and write only.
 - i. Now append another student name in **student.txt** file.
 - j. Display contents of all the files.
 - k. Show permissions and all of the file contents of **HITEC**.

Conclusion:

Lab Instructor

Fasih Ahmad